

Relační model, relační algebra

Příklad:

Uvažujme schémata $R(A, B, C)$ a $S(B, C, D, E)$ a relace na obrázku. Příklad ukazuje výsledky speciálních druhů spojení – Θ -spojení a přirozené polospojení. Znak $\dagger$ označuje spojované n-tice.

R	A	B	C
$\dagger$	8	2	3
$\dagger$	1	2	3
$\dagger$	1	1	4
	3	6	7
	3	8	9

S	B	C	D	E
$\dagger$	2	4	2	3
$\dagger$	2	3	2	3
	1	4	5	6
$\dagger$	2	3	4	7

$R [A < B] S$	A	R.B	R.C	S.B	S.C	D	E
	1	2	3	2	4	2	3
	1	2	3	2	3	2	3
	1	2	3	2	3	4	7
	1	1	4	2	4	2	3
	1	1	4	2	3	2	3
	1	1	4	2	3	4	7

$R < A < B] S$	A	B	C
	1	2	3
	1	1	4

$R <^* S$	A	B	C
	8	2	3
	1	2	3
	1	1	4

Příklad:

Vyjádřete přirozené spojení pomocí operací projekce, selekce a kart. součin.

Příklad:

Za jaké podmínky pro relace $R(A)$ a $S(B)$ platí, že $R * S = R \cap S$?

Příklad:

Jsou následující výrazy ekvivalentní?

$R(\varphi_1)(\varphi_2)$ $R(\varphi_2)(\varphi_1)$ $R(\varphi_1 \text{ and } \varphi_2)$

Příklad:

Uvažujme lékařskou praxi skupiny lékařů s relační databází se třemi relacemi o schématech:

LÉKAŘ(Č_LICENCE, JMÉNO_L, SPECIALIZACE)

PACIENT(Č_PACIENTA, JMÉNO_P, ADRESA, TELEFON, D_NAROZENÍ)

NÁVŠTĚVA(Č_LICENCE, Č_PACIENTA, TYP, DATUM, DIAGNÓZA, CENA)

kde TYP znamená typ návštěvy (domů na zavolání, do ordinace, do nemocnice apod.). Napište v relační algebře tyto požadavky:

Seznam všech registrovaných pacientů.

řešení: PACIENT

Seznam všech specializací lékařů.

řešení: LÉKAŘ[specializace]

Seznam všech orthopedů.

řešení: LÉKAŘ(specializace="orthoped")

Jména všech orthopedů.

řešení: LÉKAŘ(specializace="orthoped")[jméno_l]

Seznam všech pacientů narozených před r. 1920.

Licence všech lékařů, které navštívila Marie Nová.

Jména všech lékařů, kteří byli na návštěvě domů na zavolání.

Seznam (všechny detaily) všech navštívených pacientů.

Jména a adresy všech pacientů, kteří byli vyšetřováni Dr. Lomem 23.4.1992.

řešení: (PACIENT * NÁVŠTĚVA(datum="23.4.1992")
* LÉKAŘ(jméno_l="Lom")) [jméno_p, adresa]

Jména a adresy všech pacientů, u kterých byla určena diagnóza HIV-positivní.

Nalezni jména a specializace všech lékařů, kteří určili diagnózu vřed na dvanácterníku.

SQL – DDL a základy DML

Příklad:

Uvažujme seznam kin a seznam filmů a jejich promítání. Kino má atributy: název, adresa, počet míst, Dolby ano/ne. Film má atributy: název, rok vzniku, země vzniku, dabing ano/ne.

Uvažujte několik variant v níže uvedených kombinacích, navrhnete datové struktury bez a s použitím prázdných hodnot (NULL):

A. v každém kině se může dávat víc filmů, každý film se může dávat pouze v jednom kině.

B. v každém kině se může dávat víc filmů a každý film se může dávat ve více kinech.

1. chceme evidovat jen filmy, které jsou někde dávány

2. chceme evidovat všechny filmy

α. jeden film se v jednom kině dává vždy jen jednou

β. jeden film se v jednom kině může dávat víckrát, chceme odlišit data různých promítání

Přihlašte se k PostgreSQL (viz cvičení 1). Pro variantu B2β vytvořte tabulky a vyzkoušejte následující příkazy v jazyce SQL (doporučuji použít pro práci na serveru např. WinSCP, kde můžete snadno vytvářet a editovat skriptovací soubory, abyste nemuseli nepovedené příkazy psát stále znova celé do příkazové řádky):

- definujte všechny tabulky vč. IO, zaměřte se na použití IO sloupce a IO tabulky, všechny cizí klíče definujte tak, že při pokusu o porušení referenční integrity dojde k chybě

možné řešení:

```
CREATE TABLE Kino (  
    nazev_k    VARCHAR(50)    PRIMARY KEY,  
    adresa     VARCHAR(200)   NOT NULL,  
    mist       INT            NOT NULL,  
    dolby      BOOLEAN        DEFAULT 'f');
```

```
CREATE TABLE Film (
    nazev_f    VARCHAR(100)    PRIMARY KEY,
    rok        INT             NOT NULL CHECK(rok>1920),
    zeme       VARCHAR(20)     DEFAULT "ČR",
    dabing     BOOLEAN);
```

```
CREATE TABLE Program (
    nazev_k    VARCHAR(50)     REFERENCES Kino(nazev_k),
    nazev_f    VARCHAR(100)    REFERENCES Film(nazev_f),
    datum      DATE,
    PRIMARY KEY (nazev_k, nazev_f, datum));
```

- vložte do tabulek nějaká data (nejméně pět řádků v každé tabulce, aspoň jedno kino z Jihlavy)
- vytvořte tabulku "cizí filmy", do níž vložíte všechny filmy, které nejsou z ČR
- změňte u filmů z ČR zemi z ČR na Česko
- u všech kin zvyšte kapacitu o 20 %
- odstraňte z tabulky KINO všechna kina z Jihlavy
- odstraňte z tabulky FILM informaci o dabingu
- změňte název sloupce „počet míst“ na "kapacita"
- u sloupce země vzniku nastavte jako implicitní hodnotu (default) prázdný řetězec
- přidejte ke sloupci kapacita pojmenované IO hlídající, že kapacita>0 a zároveň kapacita<1000
- změňte IO u sloupce kapacita tak, že rozmezí nebude 1 až 999, ale 20 až 1099
- některý z cizích klíčů upravte tak, že při smazání odkazovaného řádku se smažou i odkazující
- přidejte k tabulce KINO sloupec jméno vedoucího tak, aby byl NOT NULL
- přidejte k tabulce KINO sloupec id_kina a učiňte z něj primární klíč (se všemi důsledky pro ostatní tabulky)

Vyzkoušejte v SQL následující dotazy:

- vypiš názvy kin, která mají kapacitu větší než 100 míst

řešení:

```
SELECT nazev_k
FROM Kino
WHERE mist>100;
```

- vypiš názvy filmů, které byly natočeny v USA
- vypiš filmy, které se hrají v kině Dukla
- vypiš názvy a adresy kin, kde se dává film Titanic
- vypiš názvy a adresy kin, kde se dává nějaký film z Japonska

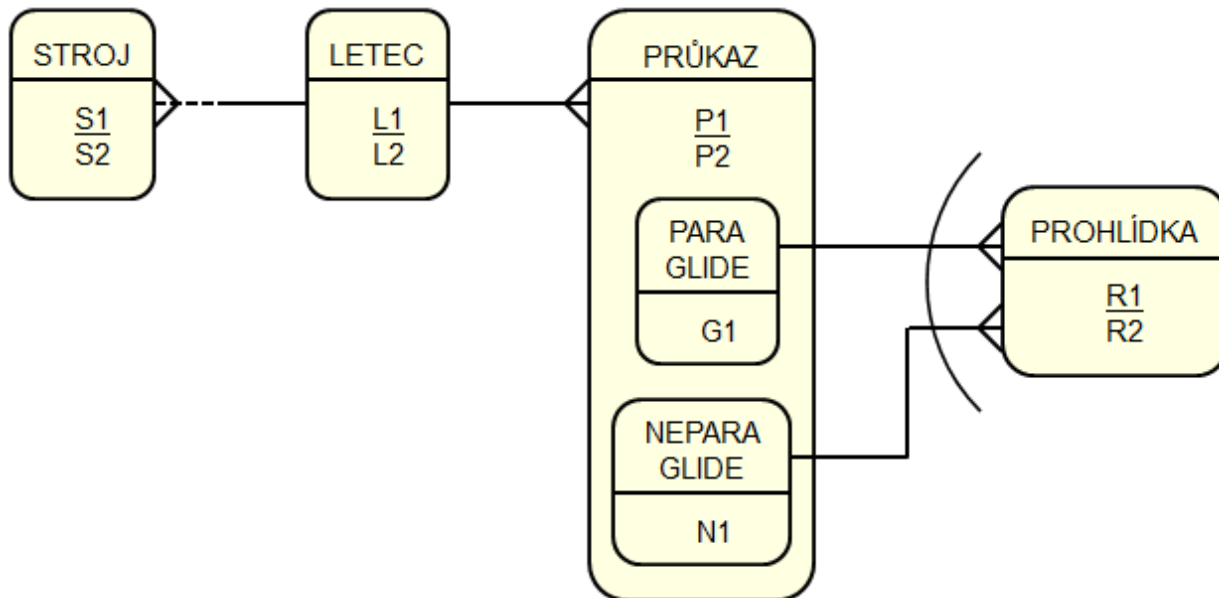
možné řešení:

```
SELECT nazev_k, adresa
FROM Kino NATURAL JOIN Program NATURAL JOIN Film
WHERE zeme='JP';
```

Transformace ER modelu do tabulek

Příklad:

Transformujte do tabulek modely dle obrázků. Tabulky запиšte v SQL, soustřeďte se na IO, datové typy není třeba řešit příliš podrobně. Komentujte Vámi zvolený způsob reprezentace vztahů M:N, ISA hierarchií, výlučných vztahů, nepovinných členství determinantů vztahů.



možné řešení (datové typy pro jednoduchost voleny všechny INT):

```
CREATE TABLE Stroj (  
    s1    INT    PRIMARY KEY,  
    s2    INT    NOT NULL,  
    l1    INT    REFERENCES Letec(l1));  
  
CREATE TABLE Letec (  
    l1    INT    PRIMARY KEY,  
    l2    INT    NOT NULL);  
  
CREATE TABLE Pukaz (  
    p1    INT    PRIMARY KEY,  
    p2    INT    NOT NULL,  
    l1    INT    REFERENCES Letec(l1));  
  
CREATE TABLE Paraglide (  
    p1    INT    PRIMARY KEY REFERENCES Pukaz(p1),  
    g1    INT    NOT NULL);  
  
CREATE TABLE Neparaglide (  
    p1    INT    PRIMARY KEY REFERENCES Pukaz(p1),  
    n1    INT    NOT NULL);  
  
CREATE TABLE Prohlidka (  
    r1    INT    PRIMARY KEY,  
    r2    INT    NOT NULL,  
    pg    INT    REFERENCES Paraglide(p1),  
    pn    INT    REFERENCES Neparaglide(p1),  
    CHECK( (pg IS NOT NULL AND pn IS NULL) OR  
           (pg IS NULL AND pn IS NOT NULL)) );
```

jiné možné řešení ISA hierarchie a vztahů s Prohlídkou:

```
CREATE TABLE Pukaz (  
    p1    INT    PRIMARY KEY,  
    p2    INT    NOT NULL,  
    g1    INT    NOT NULL,  
    n1    INT    NOT NULL,  
    typ   INT    NOT NULL,  
    l1    INT    REFERENCES Letec(l1));
```

```
CREATE TABLE Prohlídka (  
    r1    INT    PRIMARY KEY,  
    r2    INT    NOT NULL,  
    p1    INT    NOT NULL REFERENCES Pukaz(p1));
```

